

به نام خدا

پایگاه داده به عنوان یک سرویس (DBaaS)

ارائه دهندگان: سید حمیدرضا حسینی ، محمد افضل زاده نائینی

بخش اول: مقدمه

۱-۱. بیان مسئله

داده‌ها امروز به عنوان ارزشمندترین دارایی هر سازمان شناخته می‌شوند. تصمیم‌گیری مبتنی بر داده، پیش‌بینی روندهای آینده، افزایش بهره‌وری و کسب مزیت رقابتی، همگی به مدیریت درست داده‌ها وابسته هستند.

با رشد سریع حجم داده‌ها و نیاز به دسترسی پذیری بالا، مدیریت پایگاه داده به صورت سنتی (نصب و نگهداری دستی توسط تیم‌های داخلی) با چالش‌های زیر همراه است:

- پیچیدگی در راه‌اندازی و نگهداری پایگاه داده‌ها
- نیاز به نیروهای متخصص دائمی
- خطر از دست رفتن داده‌ها به دلیل خطای انسانی یا مشکلات سخت‌افزاری
- محدودیت در مقیاس پذیری

راهکاری مدرن برای رفع این چالش‌ها، پایگاه داده به عنوان سرویس (Database as a Service – DBaaS) است. در این مدل، مدیریت پایگاه داده به صورت خودکار و سرویس‌محور انجام می‌شود و سازمان‌ها می‌توانند بدون درگیری با مسائل زیرساختی، تنها روی استفاده و تحلیل داده‌ها تمرکز کنند.

۲-۱. اهداف پروژه

اهداف اصلی این پروژه عبارتند از:

- طراحی و پیاده‌سازی نمونه‌ای از سرویس DBaaS
- ایجاد زیرساخت پایگاه داده مقیاس‌پذیر و امن
- خودکارسازی فرآیندهای نصب، راه‌اندازی و مدیریت پایگاه داده
- پشتیبانی از مکانیزم‌های پیشرفته بکاپ‌گیری و بازیابی
- ارائه قابلیت مانیتورینگ و اطلاع‌رسانی لحظه‌ای
- کاهش وابستگی به نیروی انسانی متخصص برای فرآیندهای قابل خودکارسازی

بخش دوم: روش کار

۱-۲. نیازمندی‌های سیستم

سیستم طراحی شده باید نیازهای زیر را برآورده کند:

- مقیاس‌پذیری: توانایی افزایش منابع پردازشی و ذخیره‌سازی متناسب با بار کاری
- دسترسی‌پذیری بالا (High Availability): حذف نقاط شکست منفرد (SPOF)
- بکاپ‌گیری مطمئن: تهیه خودکار و دستی نسخه‌های پشتیبان + پشتیبانی از Point-in-Time Recovery
- مانیتورینگ پیشرفته: پایش وضعیت سرویس‌ها و ارسال هشدار در زمان بروز خطا
- انعطاف‌پذیری: قابلیت انتخاب سرویس متناسب با نیازهای هر سازمان

۲-۲. سناریو کاربری پروژه

1. ورود به پنل کاربری: کاربر وارد پنل مدیریت می‌شود تا سرویس نگهداری داده خود را ایجاد کند. در این پنل می‌تواند با توجه به نیازهای خاص هر سازمان سرویس را تنظیم کند.
2. انتخاب سرویس متناسب: کاربر در حین سفارش سرویس، گزینه‌ای را انتخاب می‌کند که با توجه به ابعاد داده و نیازهای کسب‌وکار او بهینه شده است. هر سرویس با در نظر گرفتن سطح پیچیدگی‌های مختلف برای نگهداری پایدار داده‌ها طراحی شده است.
3. پیاده‌سازی خودکار زیرساخت: پس از تنظیم سرویس، سیستم به‌صورت خودکار زیرساخت نگهداری داده‌ها را در سرورهای موردنیاز مستقر می‌کند. این کار از طریق یک سرویس ارکستریشن پیشرفته (Kubernetes) انجام می‌شود که اطمینان حاصل می‌کند همه نیازهای کاربر به‌صورت بهینه و اتوماتیک برآورده شود.
4. مانیتورینگ و مدیریت: پس از پیاده‌سازی، کاربر به‌صورت لحظه‌ای به مانیتورینگ وضعیت سیستم دسترسی دارد. این سیستم به کاربر امکان می‌دهد در هر لحظه از عملکرد کلاسترها، وضعیت Replication، و سطح دسترسی‌پذیری مطلع می‌شود. همچنین، در صورت بروز هرگونه خطا، سیستم به‌صورت خودکار اعلان‌ها و هشدارهای لازم را ارسال می‌کند.
5. Backup & Restore: سرویس امکان تهیه بکاپ‌های خودکار در بازه‌های زمانی دلخواه کاربر را فراهم می‌کند و همچنین کاربر می‌تواند در هر زمان بکاپ دستی ایجاد کند. در بخش بازیابی (Restore)، علاوه بر بازگردانی به یک بکاپ مشخص، قابلیت Point-in-Time Recovery نیز در صورت نیاز کاربر در دسترس است تا دیتابیس به هر لحظه دلخواه بازگردانی شود.
6. پشتیبانی و به‌روزرسانی: سرویس به‌طور مداوم توسط ارائه‌دهنده پشتیبانی می‌شود. این شامل به‌روزرسانی‌های زیرساختی، پشتیبان‌گیری روزانه و رفع مشکلات احتمالی در بازه زمانی مشخص شده توسط کاربر است.

بخش سوم: پیاده‌سازی

۳-۱. معماری سیستم

معماری پیشنهادی سیستم بر اساس مدل کلاستر پایگاه داده طراحی شده که در آن اجزای مختلف به صورت یکپارچه عمل می‌کنند تا سرویس پایدار، مقیاس پذیر و مقاوم در برابر خطا فراهم شود. دیاگرام معماری اجزای اصلی زیر را نمایش می‌دهد:

۱. کاربر

کاربر نهایی از طریق پنل مدیریتی با سیستم تعامل می‌کند. وظایف اصلی کاربر عبارت‌اند از:

- سفارش سرویس پایگاه داده متناسب با نیاز سازمان
- مشاهده وضعیت کلاستر و عملکرد سرویس
- ایجاد یا بازبینی نسخه‌های پشتیبان در صورت نیاز
- ارسال پرس‌وجوهای خواندنی و نوشتنی

۲. مدیریت اتصال و توزیع بار پایگاه داده: (PgCat – Connection Pooling)

این بخش نقش کلیدی در مدیریت ارتباط بین کاربر و پایگاه داده‌ها دارد. وظایف PgCat عبارت‌اند از:

- **Connection Pooling**: مدیریت تعداد زیاد اتصال‌های هم‌زمان و بهینه‌سازی مصرف منابع
- **Query Splitting**: تفکیک پرس‌وجوها به دو دسته خواندنی و نوشتنی؛ پرس‌وجوهای خواندنی به Replicaها هدایت می‌شوند و پرس‌وجوهای نوشتنی به Master ارسال می‌شوند
- **Load Balancing**: توزیع خودکار پرس‌وجوهای خواندنی بین Replicaهای مختلف برای افزایش سرعت پاسخ‌گویی و جلوگیری از سربار روی یک سرور
- **Sharding** (در آینده): قابلیت تقسیم داده‌ها بین چند پایگاه داده برای مقیاس‌پذیری افقی

۳. پایگاه داده اصلی (Master Database)

پایگاه داده اصلی مسئول پردازش کلیه عملیات نوشتنی (INSERT, UPDATE, DELETE) است. ویژگی‌های مهم آن:

- همگام‌سازی تغییرات با Replicaها به صورت خودکار
- تضمین یکپارچگی و پایداری داده‌ها
- تنها نقطه ورودی برای عملیات تغییر داده

۴. پایگاه‌های داده کپی (Replica Database)

Replica ها نسخه‌های همگام‌شده با پایگاه داده اصلی هستند که وظیفه پاسخ‌گویی به پرس‌وجوهای خواندنی را بر عهده دارند.

- تمامی Replica ها در حالت SYNC با Master به‌روزرسانی می‌شوند
- در صورت خرابی Master، یکی از Replica ها به‌طور خودکار یا دستی ارتقا یافته و جایگزین آن می‌شود (Failover)
- توزیع بار بین Replica ها باعث افزایش کارایی سیستم و کاهش زمان پاسخ‌گویی می‌شود

۵. سیستم پشتیبان‌گیری (Backup System)

این بخش با استفاده از ابزار WAL-G و فضای ذخیره‌سازی سازگار با 3S (مانند MinIO) طراحی شده است. ویژگی‌های آن:

- ذخیره‌سازی مداوم تغییرات پایگاه داده به کمک (WAL) Write Ahead Logs
- پشتیبانی از Point-in-Time Recovery (PITR) برای بازگردانی دیتابیس به هر لحظه دلخواه
- تضمین امنیت داده‌ها و کاهش ریسک از دست رفتن اطلاعات

۶. مانیتورینگ و اطلاع‌رسانی

- مانیتورینگ لحظه‌ای وضعیت کلاستر (Master, Replica, Backup)
- پایش سلامت سرویس‌ها، میزان منابع مصرفی (CPU, RAM, Storage) و تأخیر اتصال‌ها (Latency)
- ارسال اعلان خودکار در صورت بروز خطا یا کاهش دسترس‌پذیری

جریان داده در معماری

1. کاربر پرس‌وجوی خود را از طریق backend یا ابزار مدیریت دیتابیس ارسال می‌کند.
2. پرس‌وجو ابتدا وارد لایه PgCat می‌شود.
3. اگر پرس‌وجو نوشتنی باشد → به Master Database ارسال می‌شود.
4. اگر پرس‌وجو خواندنی باشد → به یکی از Replica ها (بر اساس الگوریتم Load Balancing) هدایت می‌شود.
5. تغییرات انجام‌شده در Master به صورت خودکار به Replica ها منتقل می‌شود.
6. سیستم Backup به‌طور مداوم تغییرات را ذخیره کرده و امکان بازیابی داده‌ها در هر زمان را فراهم می‌کند.

-
- The diagram illustrates a PostgreSQL High Availability (HA) architecture using Helm charts. The components and their interactions are as follows:
- User Network:** The entry point for traffic, connecting to the Pgcat deployment.
 - Custom Pgcat Helm Chart:** Contains a `Pgcat Deployment` and a `pgcat config file`. It also includes a `pgcat config volume` for configuration management.
 - Control Plane:** Manages the deployment, including `pgcat`, `pgcat config file`, and `pgcat config volume`.
 - Custom PostgreSQL Helm Chart:**
 - Patroni Statefulset:** The core HA component, consisting of:
 - Primary Patroni pod:** Contains `PostgreSQL`, `Patroni`, and `Wal-G` components. It is connected to `Patroni endpoints` and `Patroni endpoints`.
 - Replica Patroni pod:** Contains `PostgreSQL`, `Patroni`, and `Wal-G` components. It is connected to `Patroni endpoints` and `Patroni endpoints`.
 - Patroni Endpoints:** A set of endpoints that manage the Patroni statefulset.
 - Patroni Endpoints:** A set of endpoints that manage the Patroni statefulset.
 - Custom Minio Helm Chart:** Contains a `Minio Statefulset` and a `Minio endpoints` component, used for storing backups.
- The architecture is designed for high availability and scalability, using Helm charts for deployment and management.

بخش پنجم: جمع‌بندی و گام‌های آتی

۵-۱. جمع‌بندی

نسخه اولیه سیستم DBAAS توانست مجموعه‌ای از قابلیت‌های کلیدی شامل Backup و Replication، Failover، Query Splitting و Restore را به‌صورت کاملاً خودکار پیاده‌سازی کند. نتایج حاصل از آزمون‌ها نشان داد که این سامانه از نظر پایداری، مقیاس‌پذیری و قابلیت اطمینان عملکردی مطلوب و قابل اتکا دارد. بنابراین می‌توان این نسخه را یک نقطه آغازین محکم برای حرکت به سوی توسعه و تکامل سیستم و تبدیل آن به یک محصول تجاری در مقیاس واقعی قلمداد کرد.

۵-۲. گام‌های آتی

با توجه به نتایج به‌دست آمده در این پروژه، مسیرهای متعددی برای توسعه و ارتقای سامانه وجود دارد که برخی از مهم‌ترین آن‌ها عبارتند از:

1. افزودن Sharding برای مقیاس‌پذیری افقی

- در حال حاضر مقیاس‌پذیری سیستم عمدتاً با تکیه بر Replicaها انجام می‌شود. افزودن Sharding امکان تقسیم داده‌ها بین چند پایگاه داده مستقل را فراهم می‌کند و ظرفیت کلی سیستم را به‌صورت قابل توجهی افزایش می‌دهد.

2. توسعه رابط کاربری گرافیکی (GUI)

- پنل کاربری فعلی ساده است. می‌توان رابط گرافیکی طراحی کرد که مدیریت سرویس‌ها، مشاهده وضعیت کلاسترها، تنظیم بکاپ‌ها و مانیتورینگ را برای کاربران غیر فنی نیز آسان کند.

3. بهبود مکانیزم‌های مانیتورینگ و هشدار

- اضافه کردن داشبوردهای دقیق‌تر (مثلاً با Grafana) و امکان تنظیم سیاست‌های هشدار پیشرفته (Alert Rules) برای ادمین‌ها.
- پشتیبانی از ارسال هشدار از طریق پیامک، ایمیل یا پیام‌رسان‌ها.

4. افزودن لایه امنیتی پیشرفته

- پشتیبانی از رمزنگاری داده‌ها در حالت سکون و انتقال (Encryption at Rest & in Transit).
- کنترل دسترسی مبتنی بر نقش (RBAC) و احراز هویت چندمرحله‌ای برای کاربران.

5. افزودن سیستم صورتحساب و مدیریت منابع (Billing & Resource Management)

- طراحی ماژولی برای پایش و ثبت میزان مصرف منابع توسط هر کاربر شامل: CPU، حافظه (RAM)، فضای ذخیره‌سازی، ترافیک شبکه

- توسعه ماژول محاسبه هزینه‌ها بر اساس مدل Pay-as-you-go.

6. پشتیبانی از چند موتور پایگاه داده (Multi-DB Support)

- در نسخه فعلی تمرکز روی PostgreSQL است. در آینده می‌توان MySQL یا MariaDB را نیز به سامانه اضافه کرد تا طیف بیشتری از نیازها پوشش داده شود.